

Guía de Estudio - Pre-Parcial NestJS: Processes & Executions

Computación en Internet III - Parcial Express

Para alguien que no sabe nada de Node/NestJS. Esta guía explica qué se hizo, por qué y cómo replicarlo en 2 horas en el parcial real. Usa la misma plantilla del pre-parcial.

1. ¿Qué es cada cosa? (desde cero)

Node.js

Es JavaScript fuera del navegador. Te permite correr JavaScript en tu computador/servidor. Sin Node no hay backend JS.

- Ejecutas con node archivo.js o npm run start.

Express

Es una librería de Node para hacer servidores HTTP. Con Express dices: "cuando llegue un GET /processes, ejecuta esta función".

- NestJS usa Express por debajo (@nestjs/platform-express en package.json). No ves Express directo, pero está ahí.

TypeScript (TS)

Es JavaScript con tipos. En vez de let x = 5, escribes let x: number = 5. Te avisa de errores antes de correr.

- Todo archivo .ts se compila a .js con npm run build.

NestJS

Framework que organiza tu backend con módulos, controladores y servicios (arquitectura limpia). Te da estructura, validación, inyección de dependencias.

- Comando clave: npx nest g module processes genera boilerplate.

MongoDB + Mongoose

- MongoDB: Base de datos NoSQL. Guarda documentos JSON, no tablas. Ideal para prototipos rápidos.
- Mongoose: Librería que conecta Node con Mongo y define Schemas (forma de tus datos).
- Docker: Levanta Mongo sin instalarlo en tu PC. docker-compose.yml lo orquesta.

Docker & docker-compose

Contenedor = máquina virtual liviana. Con docker compose up -d levantas Mongo en puerto 27017 aunque no lo tengas instalado.

2. ¿Qué pide el PDF exactamente? (sin inventos)

Del Parcial Express.pdf:

Process (7 campos):

```
name: string requerido
description: string
state: enum draft | active | archived
targetTemperature: number
targetPH: number
maxDurationHours: number
createdAt: date default now
```

Execution (8 campos):

```
processId: ObjectId ref Process requerido
startDate: date requerido
```

```

endDate: date opcional
operatorUsername: string requerido
status: enum running | stopped default running
notes: string
measuredPH: number
measuredTemperature: number

```

Operaciones obligatorias:

- Process: Create, FindAll, FindById, DeleteById, UpdateById -> 5 endpoints
- Execution: Create, FindById, FindByProcessId, DeleteById, UpdateById -> 5 endpoints (¡no hay FindAll global!)

3. Estructura final del proyecto (lo que entregamos)

```

src/
--- main.ts // Arranque + ValidationPipe
--- app.module.ts // Conexión Mongo + importa módulos
--- common/pipes/parse-object-id.pipe.ts // Valida ObjectId
--- processes/
- --- schemas/process.schema.ts // Definición Mongo de Process
- --- dto/create-process.dto.ts // Validación al crear
- --- dto/update-process.dto.ts // Validación al actualizar
- --- processes.service.ts // Lógica (habla con Mongo)
- --- processes.controller.ts // Rutas HTTP /processes
- --- processes.module.ts // Junta todo
--- executions/
- --- schemas/execution.schema.ts
- --- dto/create-execution.dto.ts
- --- dto/update-execution.dto.ts
- --- executions.service.ts
- --- executions.controller.ts
- --- executions.module.ts
docker-compose.yml // Mongo
.env // URI de Mongo

```

4. Paso 0 - Infraestructura (15 min en el parcial)

4.1 `docker-compose.yml`

```

services:
  mongo:
    image: mongo:7
    container_name: mongo-bioprocess
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_INITDB_ROOT_USERNAME:-root}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_INITDB_ROOT_PASSWORD:-root}
    ports: ["${MONGO_PORT:-27017}:27017"]
    volumes: [mongo_data:/data/db]
volumes:
  mongo_data:

```

Levanta con: docker compose up -d -> verifica docker ps.

4.2 `.env`

```

MONGO_URI=mongodb://root:root@localhost:27017/bioprocess?authSource=admin
MONGO_INITDB_ROOT_USERNAME=root
MONGO_INITDB_ROOT_PASSWORD=root
MONGO_DATABASE=bioprocess
MONGO_PORT=27017
PORT=3000

```

authSource=admin es clave porque Mongo crea el usuario en admin.

4.3 `src/app.module.ts` - Conexión a Mongo

```
MongooseModule.forRootAsync({
  imports: [ConfigModule],
  inject: [ConfigService],
  useFactory: (cs: ConfigService) => ({
    uri: cs.get<string>('MONGO_URI') ?? 'mongodb://root:root@localhost:27017/bioprocess?authSource=admin',
  }),
})
```

- ConfigModule.forRoot({isGlobal:true}) lee .env.
- MongooseModule conecta. Si URI está mal, Nest no arranca.

4.4 `src/main.ts` - Validación global

```
app.useGlobalPipes(new ValidationPipe({
  whitelist: true, // borra campos no permitidos
  forbidNonWhitelisted: true, // error si mandas campo extra
  transform: true, // convierte tipos
})));
```

Sin esto, class-validator no funciona.

5. Process - Implementación exacta (25 min)

5.1 Schema `process.schema.ts`

```
@Schema({ timestamps: { createdAt: true, updatedAt: false } })
export class Process {
  @Prop({ required: true }) name: string;
  @Prop() description: string;
  @Prop({ enum: ProcessState }) state: ProcessState;
  @Prop() targetTemperature: number;
  @Prop() targetPH: number;
  @Prop() maxDurationHours: number;
  createdAt: Date; // lo pone timestamps
}
export const ProcessSchema = SchemaFactory.createForClass(Process);
```

- @Schema marca la clase como colección Mongo.
- timestamps: {createdAt:true} crea createdAt con default: now automáticamente (pide el PDF).

5.2 DTOs - Qué valida el backend

CreateProcessDto:

```
@IsString() @NotEmpty() name: string; // único requerido
@IsOptional() @IsEnum(ProcessState) state?: ProcessState;
@IsOptional() @IsNumber() targetTemperature?: number;
// ...
```

UpdateProcessDto extends PartialType(CreateProcessDto) -> todos opcionales al actualizar.

5.3 Service `processes.service.ts` - Los 5 métodos del PDF

```
create(dto) { return new this.processModel(dto).save(); }
findAll() { return this.processModel.find().exec(); }
findOne(id) {
  const doc = await this.processModel.findById(id).exec();
  if(!doc) throw new NotFoundException(`Process ${id} not found`);
  return doc;
}
update(id, dto) { return this.processModel.findByIdAndUpdate(id, dto, {new:true, runValidators:true}); }
remove(id) { return this.processModel.findByIdAndDelete(id); }
```

- findByIdAndUpdate con {new:true} devuelve el actualizado.
- Siempre NotFoundException si no existe -> Nest devuelve 404.

5.4 Controller `processes.controller.ts` - Las 5 rutas

```
@Controller('processes')
export class ProcessesController {
  @Post() create(@Body() dto: CreateProcessDto) {}
  @Get() findAll() {}
  @Get(':id') findOne(@Param('id', ParseObjectIdPipe) id: string) {}
  @Patch(':id') update(@Param('id', ParseObjectIdPipe) id: string, @Body() dto: UpdateProcessDto) {}
  @Delete(':id') remove(@Param('id', ParseObjectIdPipe) id: string) {}
}
```

- @Controller('processes') prefija todas las rutas.
- ParseObjectIdPipe valida que :id sea un ObjectId válido (24 hex).

5.5 Module

```
@Module({
  imports: [mongooseModule.forFeature([
    {name: Process.name, schema: ProcessSchema}
  ])],
  controllers: [ProcessesController],
  providers: [ProcessesService],
})
export class ProcessesModule {}
```

6. Execution - Implementación exacta (30 min)

6.1 Schema

```
@Schema()
export class Execution {
  @Prop({type: Types.ObjectId, ref: Process.name, required:true}) processId: Types.ObjectId;
  @Prop({required:true}) startDate: Date;
  @Prop() endDate: Date;
  @Prop({required:true}) operatorUsername: string;
  @Prop({enum: ExecutionStatus, default:'running'}) status: ExecutionStatus;
  @Prop() notes: string;
  @Prop() measuredPH: number;
  @Prop() measuredTemperature: number;
}
```

- ref: Process.name es la relación. No crea JOIN, solo referencia.
- default: running cumple el PDF.

6.2 DTOs

```
@IsMongoId() @IsNotEmpty() processId: string; // ObjectId en string
@IsDateString() @IsNotEmpty() startDate: string;
@IsString() @IsNotEmpty() operatorUsername: string;
@IsOptional() @IsDateString() endDate?: string;
// status, notes, measured... opcionales
```

- @IsMongoId valida ObjectId.
- @IsDateString valida "2026-08-27T10:00:00.000Z".

6.3 Service - Los 5 métodos del PDF + validación de FK

```
async create(dto) {
  const exists = await this.processModel.exists({_id: dto.processId});
  if(!exists) throw new NotFoundException(`Process ${dto.processId} not found`);
  return new this.executionModel({
    ...dto,
    processId: new Types.ObjectId(dto.processId),
    startDate: new Date(dto.startDate),
  }).save();
}
```

```

}
findOne(id) { /* findById */ }
findByProcessId(pid) {
  const exists = await this.processModel.exists({_id: pid});
  if(!exists) throw new NotFoundException(...);
  return this.executionModel.find({processId: new Types.ObjectId(pid)}).exec();
}
update(id, dto) { /* valida processId si viene, convierte fechas */ }
remove(id) { return findByIdAndDelete }

```

- Clave: siempre verificar que processId exista antes de crear/buscar. Si no, 404. Esto es lo que más califican.

6.4 Controller - Orden importa

```

@Controller('executions')
export class ExecutionsController {
  @Post() create(@Body() dto: CreateExecutionDto) {}
  @Get('process/:processId') findByProcessId(...) {} // ¡ANTES que :id!
  @Get(':id') findOne(@Param('id', ParseObjectIdPipe) id: string) {}
  @Patch(':id') update(...) {}
  @Delete(':id') remove(...) {}
}

```

Si pones @Get(':id') antes que process/:processId, Nest trata process como un id y falla.

6.5 Module

```

MongoModule.forFeature([
  {name: Execution.name, schema: ExecutionSchema},
  {name: Process.name, schema: ProcessSchema}, // necesita Process para validar FK
])

```

7. Flujo de una petición (para entender Nest)

1. Cliente: POST /processes {name: "Fermentación"}
2. Controller @Post() -> lee @Body() -> CreateProcessDto
3. ValidationPipe -> valida @IsNotEmpty, @IsEnum -> si falla 400
4. Controller llama a Service.create(dto)
5. Service -> this.processModel(dto).save() -> MongoDB
6. Mongo guarda documento con _id, createdAt
7. Service retorna documento -> Controller -> JSON al cliente 201

8. Comandos que debes memorizar (parcial 2h)

```

# 1. Preparación (2 min)
cp .env.example .env
docker compose up -d
docker ps # debe mostrar mongo-bioprocess
npm install # si faltan deps

# 2. Desarrollo
npm run start:dev # modo watch
npm run build     # compilar
npm run lint      # eslint

# 3. Probar (sin Postman, con curl)

```

```
curl -X POST http://localhost:3000/processes -H "Content-Type: application/json" \
  -d '{"name": "Fermentación", "state": "draft", "targetTemperature": 30}'

curl http://localhost:3000/processes
curl http://localhost:3000/processes/<id>
curl -X PATCH http://localhost:3000/processes/<id> -H "Content-Type: application/json" -d '{"state": "active"}'
curl -X DELETE http://localhost:3000/processes/<id>

curl -X POST http://localhost:3000/executions -H "Content-Type: application/json" \
  -d '{"processId": "<id>", "startDate": "2026-08-27T10:00:00.000Z", "operatorUsername": "jmrojas"}'
curl http://localhost:3000/executions/process/<processId>
```

9. Errores comunes y cómo evitarlos

Error	Causa	Solución
-----	-----	-----
`404 Process not found`	al crear Execution `processId` no existe	Crea primero un Process y copia su `_id`
`400 Invalid ObjectId`	ID mal formado (no 24 hex)	Usa el `_id` que te devuelve Mongo, no inventes
`400 status must be one of...`	Mandas `state: "activo"`	Usa exactamente `draft`, `active`, `archived`
`Nest cannot resolve dependencies`	Olvidaste importar `mongooseModule.forFeature`	Revisa `imports` del Module
`Validation failed`	Mandas campo extra `foo` `whitelist: true` lo bloquea; quita el campo	
Mongo no conecta	`MONGO_URI` sin `authSource=admin`	Verifica `.env`
`process/:id` captura Execution	Orden mal en controller	Pon `process/:processId` antes que `:id`

10. Plan de 2 horas para el parcial real (simulacro)

```
00:00-00:15 Paso 0: docker up, .env, app.module, main.ts, prueba Hello World
00:15-00:40 Paso 1: Process schema + DTO + service + controller + test con curl
00:40-01:10 Paso 2: Execution schema + DTO + service + controller + test FK
01:10-01:25 Paso 3: Validar ObjectId, probar errores 400/404
01:25-01:40 Paso 4: Limpiar, npm run build, probar docker down/up
01:40-02:00 Pulmón: README, commits, Postman
```

Tip de commits (Conventional Commits):

```
git checkout -b feature/process
git commit -m "feat(process): add schema, dto and crud"
git checkout -b feature/execution
git commit -m "feat(execution): add schema and findByProcessId"
```

11. Qué te van a calificar (según el PDF)

- [] Estructura clara (src/processes, src/executions) - no todo en un archivo
- [] 10 operaciones exactas (5+5) funcionando
- [] Validación de required y enum
- [] Relación processId con 404 si no existe
- [] Uso de MongoDB y Docker (no Postgres)
- [] createdAt automático y status default running
- [] Manejo de errores 400/404 (no 500)
- [] Código en TypeScript con class-validator

12. Ejercicios para practicar (hazlos sin mirar)

1. Crea un Process con name: "Cerveza artesanal", state: "active", targetPH: 4.5. Verifica que createdAt se genere.
2. Intenta crear sin name -> debe dar 400.

3. Crea un Execution con processId inventado -> debe dar 404.
 4. Crea 2 Executions del mismo Process -> GET /executions/process/:id debe devolver 2.
 5. Actualiza un Process a archived y verifica GET /processes/:id.
 6. Borra un Process y verifica que sus Executions sigan (o no, según decidas). Explica por qué.
-

13. Glosario rápido

- DTO: Data Transfer Object. Define qué puede enviar el cliente. Si no está en el DTO, no entra.
 - Schema: Forma de la colección Mongo.
 - Pipe: Validador de parámetros (ej: ParseObjectIdPipe).
 - Service: Donde está la lógica. No sabe de HTTP.
 - Controller: Donde están las rutas HTTP. Llama al Service.
 - Module: Empaqueta Schema+Service+Controller.
 - InjectModel: Inyección de dependencia -> Nest te da el modelo Mongo sin hacer new.
-

14. Qué se hizo en este pre-parcial (para tu exposición)

- Se migró docker-compose.yml de Postgres a Mongo 7 y app.module.ts de TypeORM a Mongoose.
- Se creó ParseObjectIdPipe para validar ObjectId.
- Se implementó Process con 5 endpoints y Execution con 5 endpoints, con validación de FK.
- Se añadió ValidationPipe global en main.ts.
- Se verificó con curl que todos los CRUD devuelven 201/200/404/400 correctos.
- npm run build y npm run lint pasan.

Archivos clave modificados: docker-compose.yml, .env.example, src/app.module.ts, src/main.ts + 12 archivos nuevos en src/processes y src/executions.

Diferencia NestJS vs NextJS: NextJS es para frontend React con SSR. NestJS es para backend (lo que usamos). No los confundas en el parcial.

15. Para profundizar (cuando tengas tiempo)

- Docs oficiales: <https://docs.nestjs.com/techniques/mongodb>
- Mongoose: <https://mongoosejs.com/docs/guide.html>
- Repasa class-validator decoradores: @IsString, @IsEnum, @IsMongooseId, @IsDateString, @IsOptional.
- Practica levantar Mongo sin Docker: mongosh mongodb://localhost:27017 -> db.processes.find().

¡Éxitos en el parcial! Con esta guía y los 10 endpoints funcionando, ya tienes el 90% del examen.